App native: Windows, Linux, macOS, Android, iOS

Corso di Frontend Development – Modulo 05

Federico Giuggioloni

ITS – Corso di Frontend Development

Anno formativo 2026/2027

Contenuto del modulo

1. App native: Windows, Linux, macOS, Android, iOS
1.1 Perché tornare al nativo
1.2 Architettura comune
1.3 Apple: iOS e macOS con SwiftUI
1.4 Android con Kotlin e Jetpack Compose
1.5 Windows
1.6 Linux
1.7 Cross-platform nativo
1.8 Guidare un agente su una piattaforma nativa

Sommario del modulo

1. App native: Windows, Linux, macOS, Android, iOS
1.1 Perché tornare al nativo
1.2 Architettura comune
1.3 Apple: iOS e macOS con SwiftUI
1.4 Android con Kotlin e Jetpack Compose
1.5 Windows
1.6 Linux
1.7 Cross-platform nativo
1.8 Guidare un agente su una piattaforma nativa

Che cosa vuol dire “nativo”

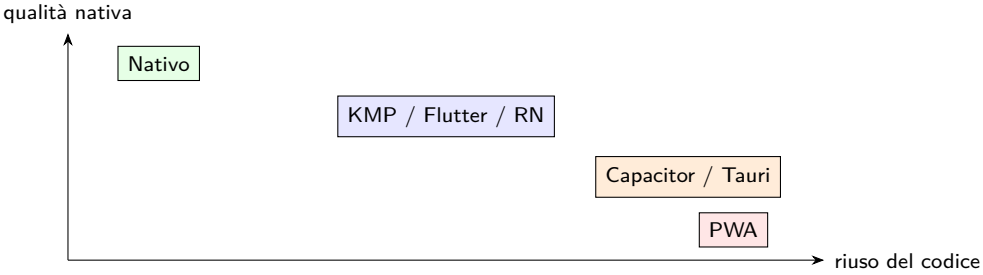
- Un’app scritta con il **linguaggio e il toolkit UI della piattaforma**: Swift/SwiftUI su Apple, Kotlin/Compose su Android, C#/WinUI su Windows, GTK o Qt su Linux.
- Niente browser, niente webview: i widget sono quelli del sistema, disegnati dal sistema.
- L’app è compilata per il processore: parte in decimi di secondo, usa poca memoria.
- Ha accesso a *tutto*: sensori, background, widget, Siri/Assistant, Watch, Auto, accessibilità di sistema.

I vantaggi, concretamente

- **Prestazioni**: liste da 10.000 articoli fluide, animazioni a 120 Hz, avvio istantaneo.
- **Batteria**: niente motore JS che gira; il sistema ottimizza per voi.
- **Integrazione**: widget, notifiche ricche, condivisione, scorciatoie, drag and drop tra app, menu nativi.
- **Accessibilità**: VoiceOver/TalkBack/Narrator funzionano subito e bene.
- **Feeling**: gesti, transizioni, tastiera, selezione testo: tutto "giusto" senza sforzo.
- **Offline e background**: sincronizzazione, geofencing, Bluetooth in background.
- **Hardware**: lettori di codici a barre, stampanti di etichette, bilance, cassetti portasoldi.
- **Sicurezza**: sandbox del sistema, keychain, biometria integrate.

Il prezzo che si pagava (fino a ieri)

- **Un codice per piattaforma**: 3–5 codebase da scrivere, testare, mantenere in sincronia.
- **Un linguaggio per piattaforma**: Swift, Kotlin, C#, e persone che li conoscono.
- Ogni funzionalità nuova costa N volte; ogni bug di logica va corretto N volte.
- Le aziende piccole non potevano permetterselo: da qui Electron, Capacitor, React Native, Flutter.



Cosa è cambiato nel 2026: gli agenti di programmazione

- Un agente (Claude Code, Codex, Copilot agent, Cursor, Gemini CLI) scrive, compila, esegue i test e corregge **da solo**, in un linguaggio che voi *non* conoscete a fondo.
- Riprodurre la pagina del catalogo di minicommerce in tre toolkit diversi è un lavoro **meccanico e ben specificato**: esattamente ciò in cui gli agenti eccellono.
- Il costo marginale della "N-esima piattaforma" è crollato. Il costo che resta è **specificare bene** e **verificare** su dispositivo.
- Conseguenza: il compromesso "web dentro una webview per risparmiare" non è più scontato. Per molte app, il nativo torna a essere la scelta economica.

Attenzione

"Economico" non vuol dire "gratis": serve qualcuno che sappia *giudicare* un'app nativa. È il vostro ruolo.

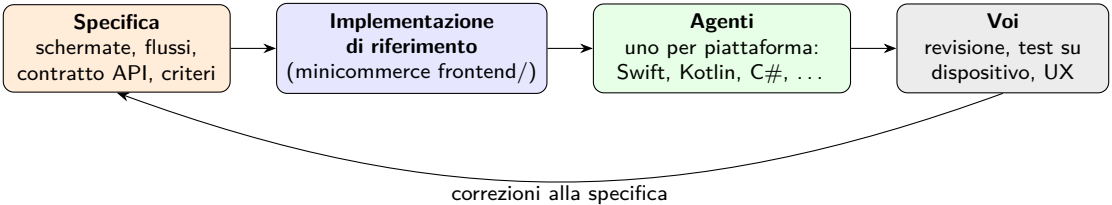
Quattro app native per minicommerce

- **Android, magazzino**: lettore di codici a barre per trovare l'articolo, rettifica di `stock`, foto dell'articolo con la fotocamera. Usata in piedi, con una mano, su un telefono economico.
- **iOS, agente di vendita**: catalogo e scheda articolo, prezzi e disponibilità, carrello del cliente per preparare l'ordine, funziona anche senza rete in capannone.
- **Windows, cassa**: lettore USB (che "digita" il codice), stampante di etichette, scontrino, tastiera senza mouse.
- **Linux, chiosco in negozio**: schermo fisso che mostra il catalogo a schermo intero, nessuna tastiera, riavvio automatico, aggiornamento via Flatpak.

Il filo comune

Tutte e quattro parlano con lo **stesso backend Express sulla porta 3001**. Il server non cambia: cambia solo il client. Stesso campo `isAdmin`, quindi stessi due ruoli: l'admin gestisce il catalogo, il cliente usa carrello e ordine.

Il nuovo flusso di lavoro

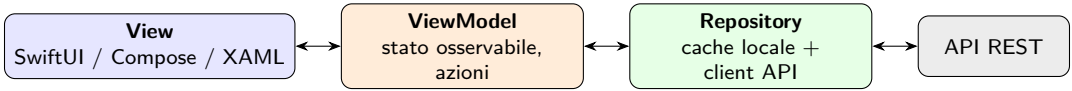


- La **specifica** è il vero codice sorgente: ciò che non c'è scritto, l'agente lo inventa.
- `frontend/src/pages/CatalogPage.tsx` resta l'implementazione di riferimento: è quella che capite meglio e da cui l'agente "traduce".
- Cosa condividere davvero: il riferimento completo delle rotte in `docs/API.md`, le convenzioni di `AGENTS.md`, testi, casi di prova. *Non* il codice UI.

Cosa resta difficile (e vostro)

- **Giudicare la UX**: un'app Android che si comporta come iOS è sbagliata anche se compila.
- **Testare su dispositivi reali**: il lettore di codici a barre e la stampante di etichette non esistono nell'emulatore.
- **Le linee guida degli store**: revisione Apple, privacy, permessi (fotocamera!). Un agente non sa cosa vi rifiuteranno.
- **Verificare le API usate**: gli agenti a volte usano API deprecate o inventate. Compilare è il primo test, non l'ultimo.
- **La parità tra piattaforme**: una checklist per funzionalità, aggiornata a ogni release.

Lo stesso disegno su tutte le piattaforme



- Il pattern **MVVM** è la lingua franca: SwiftUI, Compose e WinUI lo assumono.
- È lo stesso schema del frontend web: pagina ↔ query/stato ↔ `lib/api.ts`.
- Il `Repository` nativo è il gemello di `lib/api.ts`: un solo punto che conosce gli URL, il `Bearer` e i tipi.
- I **design token** (colori, spaziature) si esportano in un JSON e ogni piattaforma li legge.

Il contratto genera i client (esercizio)

- – Nel repository di minicommerce **non c'è un `openapi.yaml`**: il contratto è scritto in prosa in `docs/API.md` (e in forma ridotta nella tabella del `README.md`).
- + Tradurlo in OpenAPI è il primo esercizio: una ventina di rotte, cinque schemi (`Product`, `Category`, `AuthUser`, `Cart`, ordine finto), i parametri di `GET /api/products`. Poi:

```
# openapi.yaml nella root del repository; nessuna installazione globale, solo npx
OGEN="npx @openapitools/openapi-generator-cli generate -i openapi.yaml"
$OGEN -g swift6           -o clients/swift
$OGEN -g kotlin           -o clients/kotlin
$OGEN -g csharp           -o clients/csharp
$OGEN -g typescript-axios -o clients/ts     # anche per il web!
```

- Modelli tipizzati, un metodo per ogni endpoint, autenticazione gestita. Se il contratto cambia, si rigenera e **il compilatore** vi dice cosa si è rotto in ogni app.
- Alternativa più leggera (quella delle prossime slide): scrivere a mano i modelli `Codable`/`@Serializable` e usare il client HTTP standard.

Il mondo Apple in una slide

- Linguaggio **Swift**, UI **SwiftUI** (dichiarativa, molto simile a React nel modo di pensare), IDE **Xcode** (solo macOS).
- Un solo progetto può compilare per iOS, iPadOS, macOS, watchOS, visionOS: **multipiattaforma Apple** con lo stesso codice UI.
- Concorrenza moderna con `async/await` e `actor`; rete con `URLSession`; JSON con `Codable`.
- Anteprime live in Xcode: si vede la UI cambiare mentre si scrive.
- Distribuzione: TestFlight $\rightarrow$ App Store; su Mac anche fuori dallo store con notarizzazione.

SwiftUI: i tipi di minicommerce e il login

```swift
struct Product: Codable, Identifiable {
    let id: Int
    var name: String; var category: String; var brand: String
    var price: Double; var stock: Int
}
struct AuthUser: Codable { let id: Int; let email: String; let name: String; let isAdmin: Bool }
struct LoginResponse: Codable { let token: String; let user: AuthUser }

actor ApiClient {
    let base = URL(string: "http://192.168.1.10:3001/api")!   // configurabile, non cablato
    var token: String?

    func login(email: String, password: String) async throws -> AuthUser {
        var req = URLRequest(url: base.appending(path: "auth/login"))
        req.httpMethod = "POST"
        req.setValue("application/json", forHTTPHeaderField: "Content-Type")
        req.httpBody = try JSONEncoder().encode(["email": email, "password": password])
        let (data, _) = try await URLSession.shared.data(for: req)
        let res = try JSONDecoder().decode(LoginResponse.self, from: data)
        token = res.token            // in produzione: Keychain, non UserDefaults
        return res.user
    }
}
```

SwiftUI: la chiamata al catalogo

```swift
extension ApiClient {
    // GET /api/products?search=&category=&sort=name&order=asc
    func products(search: String, category: String?) async throws -> [Product] {
        var c = URLComponents(url: base.appending(path: "products"), resolvingAgainstBaseURL: false)!
        var q = [URLQueryItem(name: "sort", value: "name"), URLQueryItem(name: "order", value: "asc")]
        if !search.isEmpty { q.append(URLQueryItem(name: "search", value: search)) }
        if let category { q.append(URLQueryItem(name: "category", value: category)) }
        c.queryItems = q
        var req = URLRequest(url: c.url!)
        if let token { req.setValue("Bearer \(token)", forHTTPHeaderField: "Authorization") }
        let (data, resp) = try await URLSession.shared.data(for: req)
        guard (resp as? HTTPURLResponse)?.statusCode == 200 else { throw ApiError.http }
        return try JSONDecoder().decode([Product].self, from: data)
    }
}
```

Notate: tipi, `async/await`, decodifica JSON, gli stessi parametri di query del web. Se leggete TypeScript, leggete Swift.

SwiftUI: ViewModel e lista articoli

```swift
@Observable final class CatalogViewModel {
    var products: [Product] = []; var loading = false; var error: String?
    var search = ""; var category: String?
    let api: ApiClient                          // init omesso

    func load() async {
        loading = true; defer { loading = false }
        do { products = try await api.products(search: search, category: category) }
        catch { error = "Errore di rete" }
    }
}

struct CatalogView: View {
    @State var vm: CatalogViewModel
    var body: some View {
        List(vm.products) { p in
            VStack(alignment: .leading) {
                Text(p.name)
                Text("\(p.brand) - \(p.category)").font(.caption).foregroundStyle(.secondary)
            }
            .badge(p.stock > 0 ? "\(p.stock) pz" : "Esaurito")
        }
        .searchable(text: $vm.search)               // ricerca di sistema, gratis
        .task(id: vm.search) { await vm.load() }    // come useEffect, ma cancellabile
        .refreshable { await vm.load() }            // pull-to-refresh gratis
        .navigationTitle("Catalogo")
    }
}
```

Cosa ottenete gratis su Apple

- `List`: scorrimento, selezione, swipe actions, ricerca (`.searchable`), pull-to-refresh.
- `NavigationStack`: catalogo $\rightarrow$ dettaglio articolo con gesto indietro e stato ripristinabile.
- Dark mode, Dynamic Type (font grandi), VoiceOver, formattazione valuta locale, iPad multi-finestra.
- Keychain per il token JWT, Face ID con due righe, notifiche push via APNs, widget con WidgetKit.
- Lo stesso codice su Mac: barra dei menu, scorciatoie, finestre multiple.

Per l'agente di vendita

Login o registrazione cliente, catalogo con ricerca e filtro categoria, scheda articolo, carrello lato server, cache locale per il capannone senza rete: un pomeriggio con un agente, un Mac e un iPhone.

Il mondo Android in una slide

- Linguaggio **Kotlin**, UI **Jetpack Compose** (dichiarativa, funzioni `@Composable`), IDE **Android Studio** (Windows, Linux, macOS).
- Architettura raccomandata da Google: ViewModel + `StateFlow` + Repository; **Hilt** per la dependency injection.
- Rete: **Retrofit** o **Ktor**; JSON con `kotlinx.serialization`; immagini con Coil; DB locale con Room.
- **Material 3**: componenti pronti, tema dinamico dai colori del wallpaper.
- Distribuzione: Google Play (AAB), ma anche APK diretto o MDM aziendale: per l'app di magazzino è la via normale.

Compose: modello, API e ViewModel

```
@Serializable data class Product(val id: Int, val name: String, val category: String,
                                 val brand: String, val price: Double, val stock: Int)

interface MinicommerceApi {
    @POST("auth/login") suspend fun login(@Body body: LoginRequest): LoginResponse
    @GET("products") suspend fun products(@Query("search") search: String?,
                                          @Query("category") category: String?,
                                          @Query("sort") sort: String = "name"): List<Product>
    @PUT("products/{id}") suspend fun update(@Path("id") id: Int, @Body body: ProductInput): Product
}

@HiltViewModel
class CatalogViewModel @Inject constructor(private val api: MinicommerceApi) : ViewModel() {
    private val _state = MutableStateFlow(CatalogState())
    val state: StateFlow<CatalogState> = _state.asStateFlow()

    fun load(search: String? = null, category: String? = null) = viewModelScope.launch {
        _state.update { it.copy(loading = true, error = null) }
        runCatching { api.products(search, category) }
            .onSuccess { p -> _state.update { it.copy(products = p, loading = false) } }
            .onFailure { _state.update { it.copy(error = "Errore di rete", loading = false) } }
    }
}
```

Compose: la lista del catalogo

```
@Composable
fun CatalogScreen(vm: CatalogViewModel = hiltViewModel()) {
    val state by vm.state.collectAsStateWithLifecycle()
    LaunchedEffect(Unit) { vm.load() }

    Scaffold(topBar = { TopAppBar(title = { Text("Catalogo") }) },
             floatingActionButton = {
                 FloatingActionButton(onClick = { scanBarcode() }) {
                     Icon(Icons.Default.QrCodeScanner, contentDescription = "Scansiona codice")
                 }
             }
    ) { padding ->
        if (state.loading) LinearProgressIndicator(Modifier.fillMaxWidth())
        LazyColumn(Modifier.padding(padding)) {
            items(state.products, key = { it.id }) { p ->
                ListItem(headlineContent = { Text(p.name) },
                         supportingContent = { Text("${p.brand} - ${p.category}") },
                         trailingContent = { Text(if (p.stock > 0) "${p.stock} pz" else "Esaurito") })
            }
        }
    }
}
```

`LazyColumn` = lista virtualizzata; `items(key=)` = la `key` di React; `LaunchedEffect` = `useEffect`.
Stesse idee, sintassi diversa: è per questo che un agente le traduce bene.

Cosa ottenete gratis su Android

- Material 3 con tema dinamico, dark mode, TalkBack, dimensioni carattere di sistema.
- **ML Kit Barcode Scanning** e CameraX: lettore di codici a barre e foto dell'articolo in poche righe.
- `WorkManager` per rimandare al server le rettifiche di `stock` fatte offline; `Room` per il catalogo locale.
- Permessi granulari (fotocamera), biometria (`BiometricPrompt`), `EncryptedSharedPreferences` per il token JWT.
- Compose gira anche su desktop e iOS con Compose Multiplatform (prossima sezione).

Confronto con il modulo 3

La stessa foto che lì scattavamo con Capacitor qui si scatta con CameraX: identico il risultato, diverso il costo di ingresso. Oggi l'immagine finisce nel database come data URL base64; l'endpoint di upload multipart resta un esercizio, qui come sul web.

Il mondo Windows in una slide

- Linguaggio **C#** (.NET), UI **WinUI 3** con XAML (Windows App SDK): l'aspetto di Windows 11 (Fluent), Mica, dark mode.
- Alternative vive: **WPF** (enorme base installata nelle aziende), **.NET MAUI** (Windows + macOS + mobile), **Avalonia** (cross-platform, anche Linux), **Uno Platform**.
- IDE: Visual Studio (Community gratuito) o VS Code con C# Dev Kit.
- Pattern: MVVM con `CommunityToolkit.Mvvm` (`[ObservableProperty]`, `[RelayCommand]`).
- Distribuzione: **MSIX** (Microsoft Store o sideload), oppure installer classico; `winget`.

WinUI 3: ViewModel con il toolkit MVVM

```
public record ProductDto(int Id, string Name, string Category, string Brand,
                         double Price, int Stock);

public partial class CatalogViewModel : ObservableObject
{
    private readonly HttpClient _http;          // BaseAddress = http://192.168.1.10:3001/api/
    public ObservableCollection<ProductDto> Products { get; } = new();
    [ObservableProperty] private bool _loading;
    [ObservableProperty] private string? _error;
    [ObservableProperty] private string _search = "";

    public CatalogViewModel(HttpClient http) => _http = http;

    [RelayCommand]
    public async Task LoadAsync()
    {
        Loading = true; Error = null;
        try {
            var url = $"products?sort=name&order=asc&search={Uri.EscapeDataString(Search)}";
            var items = await _http.GetFromJsonAsync<List<ProductDto>>(url) ?? new();
            Products.Clear(); foreach (var p in items) Products.Add(p);
        } catch (HttpRequestException) { Error = "Errore di rete"; }
        finally { Loading = false; }
    }
}
```

WinUI 3: la vista in XAML

```
<Page x:Class="Minicommerce.CatalogPage" xmlns="http://schemas.microsoft.com/winfx/2006/xaml/presentation">
  <Grid RowDefinitions="Auto,Auto,*">
    <CommandBar Grid.Row="0">
      <AppBarButton Icon="Add" Label="Nuovo articolo" />
      <AppBarButton Icon="Print" Label="Stampa etichetta" />
      <AppBarButton Icon="Refresh" Label="Aggiorna" Command="{x:Bind ViewModel.LoadCommand}" />
    </CommandBar>
    <!-- il lettore USB "digita" il codice qui dentro e poi invia Enter -->
    <TextBox Grid.Row="1" PlaceholderText="Codice o nome articolo"
             Text="{x:Bind ViewModel.Search, Mode=TwoWay}" />
    <ProgressBar Grid.Row="1" IsIndeterminate="True" Visibility="{x:Bind ViewModel.Loading, Mode=OneWay}" />
    <ListView Grid.Row="2" ItemsSource="{x:Bind ViewModel.Products}">
      <ListView.ItemTemplate>
        <DataTemplate x:DataType="local:ProductDto">
          <StackPanel Orientation="Horizontal" Spacing="12">
            <TextBlock Text="{x:Bind Name}" Width="260" />
            <TextBlock Text="{x:Bind Category}" Opacity="0.6" Width="120" />
            <TextBlock Text="{x:Bind Price}" />
            <TextBlock Text="{x:Bind Stock}" Opacity="0.6" />
          </StackPanel>
        </DataTemplate>
      </ListView.ItemTemplate>
    </ListView>
  </Grid>
</Page>
```

Cosa ottenete gratis su Windows

- `x:Bind` = binding compilato e tipizzato: l'equivalente delle props, verificato dal compilatore.
- Fluent Design (Mica, acrilico), dark mode, alto contrasto, Narrator, navigazione da tastiera completa: alla cassa il mouse non si usa.
- Stampa (anche su stampanti di etichette) e lettori USB HID: il sistema li espone, non serve un driver vostro.
- `Windows.Security.Credentials.PasswordVault` per il token; Windows Hello per la biometria.
- Aggiornamenti automatici via MSIX senza scrivere un updater; notifiche toast con azioni.

Il mondo Linux in una slide

- Due toolkit principali: **GTK 4 + libadwaita** (GNOME) e **Qt 6 / QML** (KDE, industria, embedded).
- Linguaggi: C, ma in pratica **Rust** (gtk-rs, Relm4, Slint), **Python** (PyGObject, PySide6), C++ (Qt), Vala.
- Distribuzione universale: **Flatpak** (Flathub) e Snap; pacchetti `.deb`/`.rpm` per distro specifiche; AppImage.
- Linee guida GNOME HIG e KDE HIG: un'app Linux "giusta" segue una delle due.
- Mercato più piccolo, ma: chioschi in negozio, postazioni industriali, PA, sviluppatori, Steam Deck.

GTK 4 con Python: il chiosco del catalogo

```python
import gi; gi.require_version('Gtk', '4.0'); gi.require_version('Adw', '1')
from gi.repository import Gtk, Adw, GLib
import threading, requests

API = 'http://192.168.1.10:3001/api'

class CatalogWindow(Adw.ApplicationWindow):
    def __init__(self, app):
        super().__init__(application=app, title='minicommerce', fullscreened=True)
        self.list = Gtk.ListBox(css_classes=['boxed-list'])
        box = Gtk.Box(orientation=Gtk.Orientation.VERTICAL)
        box.append(Adw.HeaderBar()); box.append(Gtk.ScrolledWindow(child=self.list, vexpand=True))
        self.set_content(box)
        threading.Thread(target=self.load, daemon=True).start()   # rete fuori dal thread UI

    def load(self):
        products = requests.get(API + '/products', params={'sort': 'name'}, timeout=5).json()
        GLib.idle_add(self.show, products)                        # torna sul thread UI

    def show(self, products):
        for p in products:
            self.list.append(Adw.ActionRow(title=p['name'],
                                           subtitle=f"{p['brand']} - {p['category']}"))
```

La stessa idea in Rust con Relm4 (estratto)

```rust
#[derive(Debug)] enum Msg { Load, Loaded(Vec<Product>) }

#[relm4::component]
impl SimpleComponent for Catalog {
    view! {
        adw::ApplicationWindow { set_title: Some("minicommerce"),
            gtk::Box { set_orientation: gtk::Orientation::Vertical,
                adw::HeaderBar {},
                #[local_ref] product_list -> gtk::ListBox { add_css_class: "boxed-list" }
            }
        }
    }
    fn update(&mut self, msg: Msg, sender: ComponentSender<Self>) {
        match msg {
            Msg::Load => { let s = sender.clone();
                tokio::spawn(async move { s.input(Msg::Loaded(api::products(None).await)); }); }
            Msg::Loaded(items) => self.products = items,   // la view si aggiorna da sola
        }
    }
}
```

Rust + GTK produce binari da 3 MB senza runtime: ideale per Flatpak e per un chiosco che deve solo accendersi e funzionare. Lo stesso Rust del core di Tauri (modulo 2).

Flatpak: pubblicare il chiosco

File `it.its.Minicommerce.json` (manifest Flatpak, estratto):

```
{ "app-id": "it.its.Minicommerce", "runtime": "org.gnome.Platform", "runtime-version": "48",
  "sdk": "org.gnome.Sdk", "command": "minicommerce",
  "finish-args": ["--share=network", "--socket=wayland", "--socket=fallback-x11"],
  "modules": [{ "name": "minicommerce", "buildsystem": "meson",
                "sources": [{ "type": "dir", "path": "." }] }] }
```

- Sandbox con **permessi dichiarati** (come Tauri e le app mobili): rete, display, notifiche.
- Un unico pacchetto per tutte le distribuzioni; aggiornamenti dal centro software, utile su un chiosco che nessuno tocca.
- Servono anche: file `.desktop`, icona, metadata AppStream (descrizione, screenshot).

Le vie di mezzo: un codice, widget nativi (o quasi)

Tecnologia	Linguaggio	Rendering	Note
React Native / Expo	TypeScript	widget nativi	**la via naturale per chi sa React**; riusate tipi e query di minicommerce; anche web
Flutter	Dart	proprio (Impeller)	pixel identici ovunque, incluso desktop; ottime prestazioni
Kotlin Multiplatform	Kotlin	Compose Multiplatform o UI nativa	logica condivisa, UI Compose anche su iOS; spinto da Google e JetBrains
.NET MAUI	C#	widget nativi	per team .NET; Windows, macOS, iOS, Android

- Tutte producono **vere app native** (niente webview), con un solo codice.
- Il prezzo: uno strato in mezzo che va aggiornato e che a volte non espone l'ultima API della piattaforma.

React Native con Expo: il catalogo

```
npx create-expo-app@latest minicommerce-mobile --template blank-typescript
cd minicommerce-mobile && npm install && npx expo start   # QR code -> Expo Go sul telefono
```

```
import { FlatList, Text, Pressable } from 'react-native';
import { useProducts } from './lib/useProducts';   // lo STESSO hook del modulo 1
import { fetchProducts, formatPrice, type Product } from './lib/api';

export function CatalogScreen({ search }: { search: string }) {
  const { data, loading, reload } = useProducts({ search, sort: 'name', order: 'asc' });
  return (
    <FlatList data={data} keyExtractor={(p: Product) => String(p.id)} onRefresh={reload}
      refreshing={loading} renderItem={({ item }) => (
        <Pressable style={{ padding: 16, borderBottomWidth: 1 }} onPress={() => openProduct(item.id)}>
          <Text style={{ fontSize: 16 }}>{item.name}</Text>
          <Text style={{ opacity: 0.6 }}>{item.brand} - {item.category}</Text>
          <Text>{formatPrice(item.price)} - {item.stock > 0 ? item.stock + ' pz' : 'Esaurito'}</Text>
        </Pressable>
      )} />
  );
}
```

Niente HTML: `Text` e `FlatList` diventano widget nativi. Tipi, query e client API: **gli stessi file** del web.

Flutter: la stessa schermata

```
class CatalogScreen extends StatelessWidget {
  final List<Product> products;
  const CatalogScreen({super.key, required this.products});

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(title: const Text('Catalogo')),
      body: ListView.builder(
        itemCount: products.length,
        itemBuilder: (context, i) => ListTile(
          title: Text(products[i].name),
          subtitle: Text('${products[i].brand} - ${products[i].category}'),
          trailing: Text(formatPrice(products[i].price)),
          onTap: () => context.push('/products/${products[i].id}'),
        ),
      ),
    );
  }
}
```

Flutter disegna *tutto* da solo: perfetto per UI molto personalizzate e per un chiosco identico su ogni schermo, meno per chi vuole il look esatto di ogni OS.

Come scegliere nel 2026

1. **Team React, app mobile "normale"**: React Native + Expo. Riusate query, tipi, logica; UI nativa.
2. **Serve il massimo su una piattaforma** (magazzino con lettore di codici a barre, cassa con stampante): nativo puro, con l'agente per le altre piattaforme.
3. **UI molto grafica e identica ovunque** (chiosco, brand forte): Flutter.
4. **Team Kotlin/Android che deve fare iOS**: Kotlin Multiplatform.
5. **Gestionale interno con budget minimo**: PWA/Capacitor (modulo 3) restano validi.

La domanda da fare sempre

Chi mantiene questo codice tra tre anni, e con quali strumenti? Gli agenti spostano la risposta verso il nativo; il team la decide.

La specifica per l'agente

```
# SPEC-native.md (estratto)
## Obiettivo
App Android nativa (Kotlin, Jetpack Compose, Material 3) per il magazzino di minicommerce.
## Riferimenti nel repository
- docs/API.md       -> contratto completo: rotte, parametri, codici di errore
- README.md         -> funzionalita', modello dati, credenziali demo
- AGENTS.md         -> convenzioni del progetto (stile, naming, comandi)
- frontend/src/pages/CatalogPage.tsx -> implementazione di riferimento del catalogo
- frontend/src/lib/api.ts            -> tipi Product/Cart e parametri dei filtri
## Contratto API
Base URL configurabile (BuildConfig), default http://10.0.2.2:3001/api. Auth: Bearer da
POST /api/auth/login oppure /api/auth/register. Errori: JSON { "error": "..." }.
GET /api/products restituisce l'elenco completo: nessuna paginazione lato server.
## Schermate (parita' con il web)
1. Login e registrazione cliente (401 "Credenziali non valide" visibile nella UI)
2. Catalogo (GET /api/products: ricerca, filtro categoria, ordinamento nome|price|stock)
3. Scheda articolo (nome, marca, categoria, prezzo, disponibilita', descrizione)
4. Carrello cliente (/api/cart e /api/cart/items) e ordine finto (POST /api/orders)
5. Admin: crea/modifica/elimina articolo, foto da fotocamera, conferma prima di eliminare
## Requisiti non funzionali
- ViewModel + StateFlow + Repository; Retrofit + kotlinx.serialization
- Token in EncryptedSharedPreferences; nessun segreto nel repo; ./gradlew assembleDebug
## Cosa NON fare
- Niente webview. Niente endpoint inventati. Niente API deprecate (verificare con lint).
```

Il ciclo di lavoro con l'agente

1. **Specifica** (slide precedente) + accesso al repository di minicommerce come riferimento.
2. Chiedere un **piano** prima del codice: struttura del progetto, librerie, schermate in ordine.
3. Una schermata alla volta: *implementa, compila, esegui i test, fai uno screenshot*.
4. Voi: aprite l'app sull'emulatore/dispositivo, confrontate con la checklist di parità, annotate le differenze.
5. Le correzioni tornano nella **specifica**, non solo nel prompt: la specifica serve anche per iOS e Windows.
6. Revisione finale del codice: architettura rispettata? segreti? permessi minimi? endpoint realmente esistenti?

Checklist di parità tra piattaforme

Funzionale

- ☐ Login, registrazione cliente, logout, 401
- ☐ Catalogo: ricerca e filtro categoria
- ☐ Ordinamento nome/prezzo/disponibilità
- ☐ Scheda articolo completa
- ☐ Carrello cliente: aggiunta, quantità, svuota
- ☐ Ordine dimostrativo e carrello svuotato
- ☐ Admin: crea, modifica, elimina, foto
- ☐ Stati: caricamento, errore, vuoto

Piattaforma

- ☐ Segue le linee guida (HIG / Material / Fluent / GNOME)
- ☐ Dark mode e font di sistema
- ☐ Screen reader: ogni pulsante etichettato
- ☐ Prezzi formattati in euro, locale it-IT
- ☐ Token in storage sicuro
- ☐ Permessi minimi, motivati
- ☐ Build riproducibile da riga di comando

Il carrello è solo del cliente: con un token admin `GET /api/cart` risponde 403. Anche questo fa parte della parità.

Laboratorio 5

Laboratorio: minicommerce nativa su una piattaforma a scelta

1. Scegliere: Android (Kotlin/Compose, consigliata: gira su Windows), Windows (C#/WinUI 3), Linux (GTK4), iOS/macOS (solo con Mac) o React Native/Expo.
2. Scrivere `SPEC-native.md` di una pagina citando `docs/API.md` e `CatalogPage.tsx`; con un agente: login + catalogo con ricerca e filtro + scheda articolo, provati su emulatore o dispositivo. Chi arriva in fondo aggiunge il carrello del cliente.
3. Checklist di parità; correggere almeno tre difformità trovate *leggendo il codice*.
4. Relazione di una pagina: cosa l'agente ha fatto bene, cosa avete corretto, tempo impiegato rispetto al modulo 1.

Domande di verifica

Tre vantaggi concreti di un'app nativa rispetto a una webview? Cosa condividiamo tra piattaforme e cosa no? Quando *non* sceglieresti il nativo per minicommerce?